

TripNegotiator

A Multi-Agent, Cloud-Native Trip Planning System

CSCI 5411 — Advanced Cloud Architecting — Term Project Report
Summer 2026

This report documents the design, architecture, and implementation of TripNegotiator, and maps it against the six pillars of the AWS Well-Architected Framework.

1. Project Overview

TripNegotiator is a cloud-native, multi-agent system that plans trips through simulated negotiation between specialist AI agents. A traveler submits a destination, budget, trip length, interests, age, and (optionally) an origin resolved to a real place via a geocoded location picker. Four specialist agents — Itinerary, Budget, Logistics, and Booking — then negotiate over a proposed plan for up to three rounds: the Itinerary agent proposes a day-by-day plan grounded in live web search (destination highlights, weather, per-interest highlights, and a "best time to visit / practical essentials / etiquette / safety" guide); the Budget agent audits it against a live flight/lodging cost estimate; the Logistics agent checks travel advisories, explicitly fact-checked against today's date so a resolved, years-old warning is never presented as current; and the Booking agent checks visa requirements. If any agent rejects the proposal, its objection is fed back into the next round's itinerary revision. After three rounds, or once every agent approves, a Supervisor agent finalizes the plan — surfacing any unresolved objections honestly rather than hiding them.

The system runs as a genuinely serverless AWS stack: a FastAPI application (wrapped for Lambda via Mangum) behind API Gateway, orchestrating the four negotiation agents through AWS Step Functions, with DynamoDB for negotiation state and S3 for static reference data. The same FastAPI codebase also runs entirely locally (no AWS account needed) for fast iteration, switching orchestration strategy automatically based on whether a Step Functions ARN is configured in the environment.

Domain Selection Rationale

Trip planning was chosen because it naturally exercises every category the assignment asks for: a genuine multi-step pipeline (the negotiation rounds) rather than a single request/response API; a real need for external, time-sensitive grounding data (weather, advisories, prices, exchange rates) that a static reference dataset alone can't provide; and a plausible, demonstrable non-trivial UI.

2. Functional Requirements

Core Features

- Multi-agent negotiation loop (Itinerary, Budget, Logistics, Booking, Supervisor), bounded at 3 rounds.
- Live web-search grounding for every agent's reasoning (via the open-source ddgs library), with a fallback chain (ddgs → optional self-hosted SearxNG → DuckDuckGo HTML scrape → static reference JSON) so a single upstream outage degrades a round instead of crashing it.
- Date-aware travel advisories: a dated news search is preferred over a plain text search specifically so a resolved, years-old warning is not presented to the traveler as a current one.
- Itinerary pacing/intensity tailored to the traveler's age band, without overriding their explicitly selected interests.
- Location-picker autocomplete (Open-Meteo geocoding) replacing free-typed origin/destination strings, capturing ISO country codes.
- Live origin-to-destination exchange rate (Frankfurter/ECB reference rates) shown whenever both locations resolve to different countries.

- Per-agent source citations and a destination photo gallery on the finalized itinerary.
- Trip history: browse all past negotiations and their full round-by-round detail.

User Stories

- As a traveler, I want to submit my budget, trip length, interests, and age so the system proposes an itinerary genuinely tailored to all of them — not just a generic city guide with my interests mentioned in passing.
- As a traveler, I want the budget agent to catch cost overruns (including realistic flight costs from my actual origin) and revise the plan automatically.
- As a traveler, I want to see why a proposal was rejected at each round, with the live sources behind each agent's reasoning, so I can judge whether I trust the verdict.
- As a traveler, I want to view my past trip negotiations (full round-by-round history) after the fact.
- As a traveler, I want to pick my origin/destination from real place suggestions and see the current exchange rate between them if they're in different countries.
- As a traveler, I want travel advisories to reflect what's actually true today, not a years-old resolved warning presented as current.

3. Non-Functional Requirements

Requirement	Target / Approach
Scalability	Lambda scales horizontally per-invocation with no capacity planning; DynamoDB runs on-demand and billing mode,
Availability	Step Functions persists execution state independently of any single Lambda invocation; every Task state has R
Latency	Target under 30 seconds for a full 3-round negotiation; Groq's inference speed is the dominant lever, and per-fu
Durability	DynamoDB point-in-time recovery (35-day restore window) plus explicit encryption at rest; S3 versioning with life
Security	SSM SecureString for the Groq key, no hardcoded secrets, restricted CORS, S3 public-access-block on both bu
Maintainability	One Lambda handler per agent; shared utilities (web search, DynamoDB, S3, currency) avoid duplicated logic; a

4. Architecture

The two diagrams below are authored directly as SVG (not hand-drawn ASCII, which is what an earlier draft of this project's README relied on) and are checked into docs/architecture.svg and docs/sequence.svg for editing.

4.1 Component Architecture

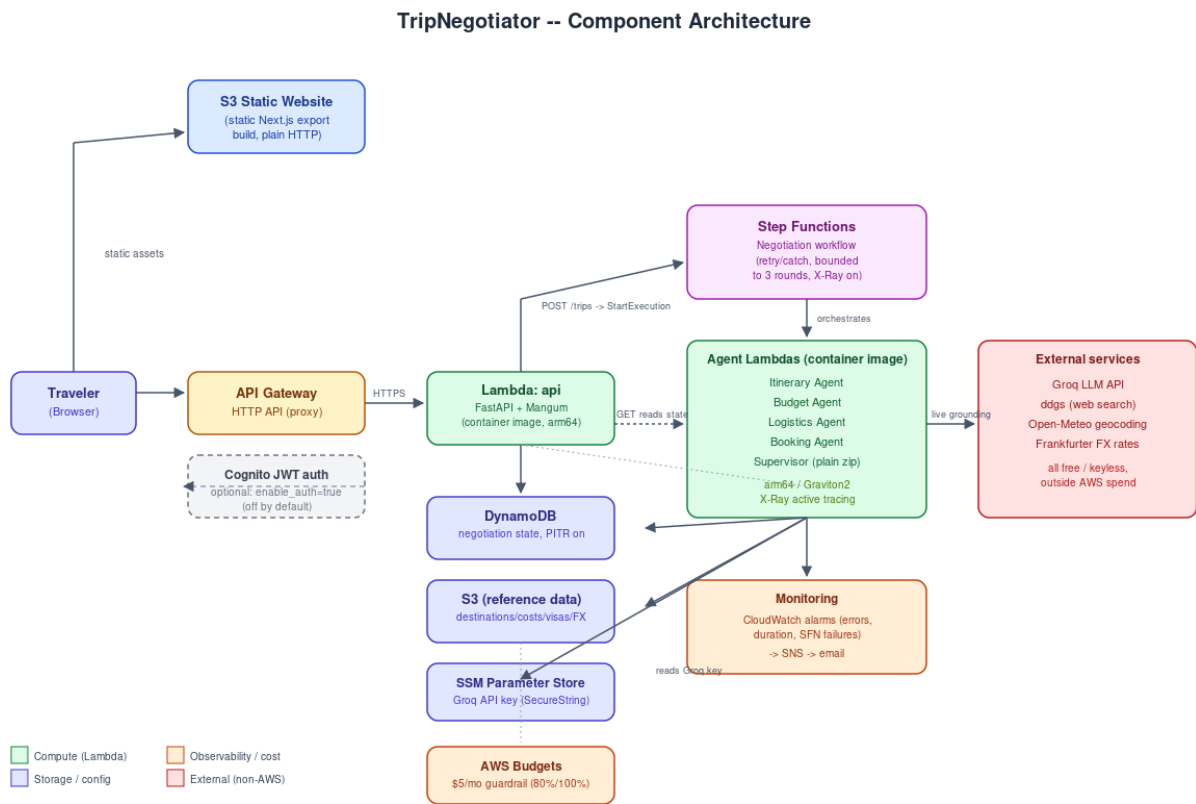


Figure 1 — Component architecture: request path, negotiation orchestration, storage, and monitoring.

4.2 Negotiation Sequence

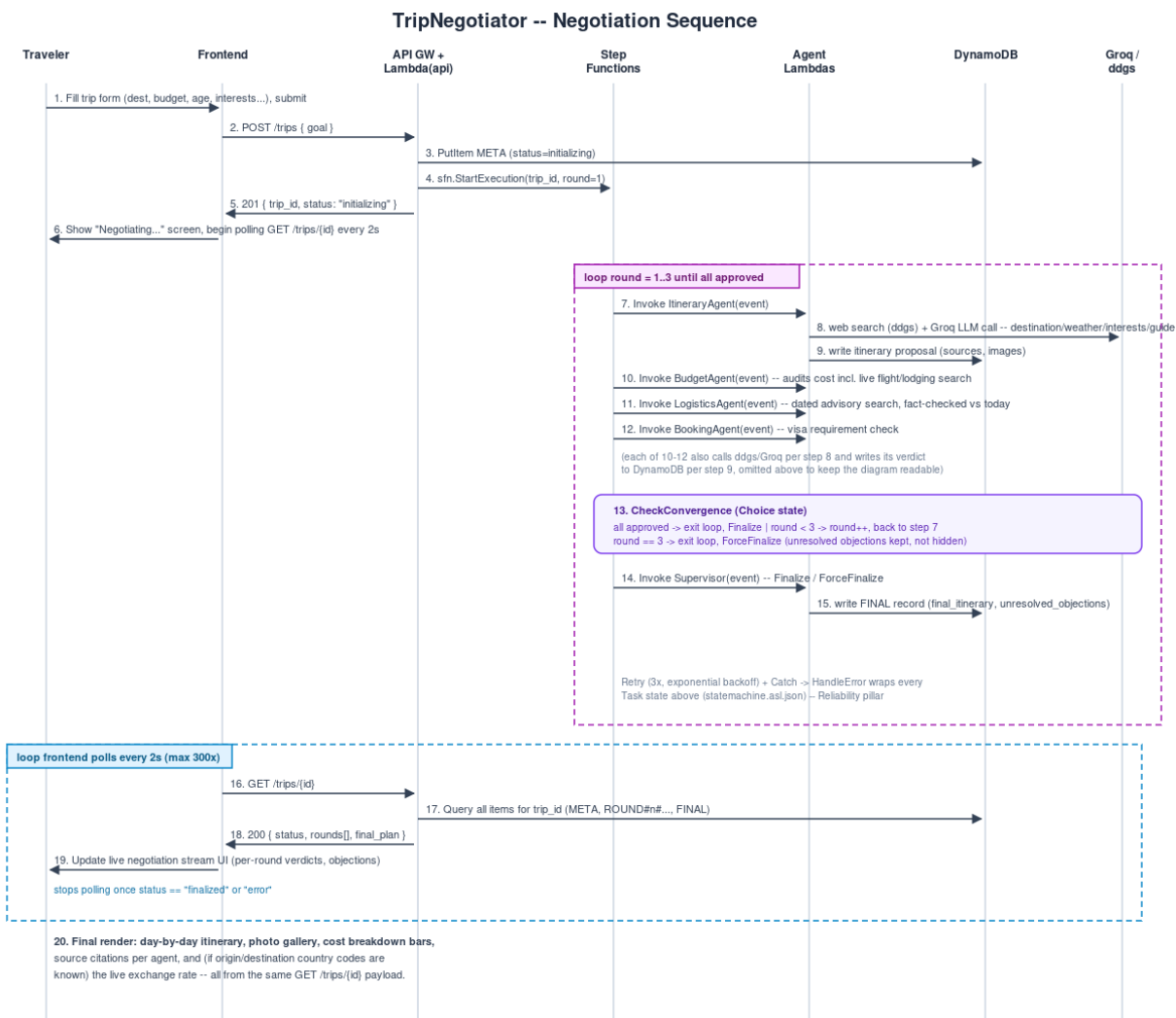


Figure 2 — Sequence diagram covering trip submission, the bounded negotiation loop, finalization, and frontend polling.

4.3 Data Flow Summary

The frontend (a statically-exported Next.js app served from a plain S3 static website endpoint -- see Section 5.8 for why this is S3-only rather than S3+CloudFront) talks to a single API Gateway HTTP API, proxied entirely to one Lambda function — the FastAPI application in `api/main.py`, wrapped for Lambda via Mangum. `POST /trips` writes the trip's metadata to DynamoDB and starts a Step Functions execution; the state machine invokes the four negotiation-agent Lambdas in sequence, each one calling out to Groq (LLM inference) and ddgs (live web search) for grounding, and each one writing its proposal/verdict back to DynamoDB. The Choice state loops back to the Itinerary agent (up to 3 times) on any rejection, or proceeds to a Supervisor Lambda that finalizes the plan. The frontend polls `GET /trips/{id}` — served by the same `api` Lambda, reading straight from DynamoDB — until the trip reaches a terminal state.

Design note: locally, the identical `api/main.py` module runs the negotiation loop in-process via a FastAPI `BackgroundTask` instead of Step Functions, selected automatically by whether `STATE_MACHINE_ARN` is set in the environment. This was a deliberate choice, not an oversight: FastAPI `BackgroundTasks` are not reliable inside a real Lambda execution environment (Lambda can freeze/recycle the process once a response is returned, which would silently kill an in-flight negotiation), so the AWS deployment must use Step Functions for orchestration while local development benefits from the simpler in-process path -- one codebase serving both, rather than two codebases drifting apart.

5. Tech Stack Justification & Trade-offs

5.1 Orchestration: AWS Step Functions (production) vs. plain sequential Lambda calls

Choice: Step Functions state machine with explicit Retry/Catch on every Task, a Choice state for the bounded negotiation loop, and two finalize branches (converged vs. round-cap).

Why: Step Functions gives durable execution state, visual debugging, and per-task retry/backoff without hand-rolling any of it. It also matches the assignment's "handle events/pipelines" requirement more directly than a monolithic Lambda would.

Trade-off: More moving infrastructure than a single Lambda; ASL (the state machine's own JSON DSL) is an extra thing to learn and keep in sync with the agent Lambdas' actual input/output shapes.

5.2 API endpoint: FastAPI + Mangum (single Lambda, proxied) vs. one Lambda per route

Choice: One container-image Lambda running the full FastAPI app behind an API Gateway proxy integration, rather than a hand-rolled router Lambda (the project's original design) or a separate Lambda per route.

Why: FastAPI gives request validation (Pydantic), automatic OpenAPI docs, and a router that can't silently drift out of sync with the frontend the way a hand-written if/elif method+path dispatcher can (and did — the original trip_init Lambda never learned about two newer routes added later). It's also the exact same code that already runs locally, so there is no separate "AWS version" of the API to maintain.

Trade-off: A single Lambda proxying everything means one function's cold start affects every route, and it's a coarser unit of IAM scoping than one-Lambda-per-route would be.

5.3 Packaging: container image vs. zip archive for Lambda

Choice: A shared Docker image (built from the Lambda Python 3.11 base image) for every function with third-party dependencies (the api Lambda: fastapi/mangum; the four agent Lambdas: ddgs), with each function overriding only its handler command. The dependency-free Supervisor Lambda stays a small zip.

Why: This project's dependencies include packages with platform-specific wheels (ddgs's transitive deps, pydantic's Rust core) that are painful to cross-compile correctly for Lambda's Linux runtime from a Windows dev machine with a plain `pip install -t`. Building inside the actual Lambda base image guarantees compatible wheels every time — the AWS-documented approach once a function's dependencies get non-trivial.

Trade-off: Container images have a larger deployment artifact and a slightly different local dev loop (rebuild + push instead of just re-zip) than plain zips; kept the dependency-free Supervisor function as a zip specifically to keep at least one fast, minimal-artifact example.

5.4 Compute architecture: arm64 (Graviton2) vs. x86_64

Choice: All Lambda functions run on arm64.

Why: AWS's own benchmarks show meaningfully lower cost and energy use per request on Graviton2 for equivalent Python workloads — directly relevant to both the Cost Optimization and Sustainability pillars (Section 6).

Trade-off: Requires building the container image specifically for `linux/arm64` (handled via `docker buildx in build-lambda-image.sh`); any dependency with architecture-specific binary wheels needs to actually publish an `arm64` wheel.

5.5 LLM: Groq vs. Amazon Bedrock

Choice: Groq's hosted LLM API (Llama 3.3 70B), called over plain `urllib` rather than a heavy SDK.

Why: Groq's inference speed is unusually fast, which matters directly for the negotiation loop's end-to-end latency (up to 3 rounds × 4 agents × an LLM call each). It also has a workable free tier for course-project-scale usage, keeping LLM cost outside the Learner Lab's 45-credit ceiling entirely.

Trade-off: Not an AWS-native service, so it doesn't count toward the "≥3 cloud services" requirement and adds an external dependency/API key to manage (via SSM SecureString). Bedrock would keep everything inside the AWS billing/IAM boundary at the cost of higher latency and Learner-Lab-credit LLM spend.

5.6 Web search grounding: ddgs vs. Tavily/Brave (paid) vs. self-hosted SearxNG

Choice: The open-source `ddgs` metasearch library (no API key, no server to run).

Why: This project explicitly wanted an open-source option that's easy to deploy on AWS. Tavily/Brave are proprietary SaaS with metered pricing; a self-hosted SearxNG instance was prototyped first but its Docker-based bot-detection limiter added deployment friction disproportionate to the value over a keyless library. SearxNG is kept as an opt-in fallback (`SEARX_HOST` env var) for anyone who does want to self-host it.

Trade-off: `ddgs` scrapes/wraps public search engines rather than using a dedicated search API, so result quality and availability are less guaranteed than a paid API's SLA — mitigated by the multi-tier fallback chain in Section 6.3's Reliability discussion.

5.7 IaC: Terraform vs. AWS CDK/CloudFormation

Choice: Terraform (HCL), with a local state backend.

Why: Terraform's provider ecosystem and plan/apply workflow are well-documented for exactly the AWS Academy Learner Lab pattern this project uses (referencing the pre-existing `LabRole` via a data source rather than creating IAM roles, which Learner Lab restricts).

Trade-off: A local state backend (not S3-backed remote state) means state isn't shared across machines/CI runs automatically — acceptable for a single-developer course project, but a real production setup would use a remote backend with locking.

5.8 Frontend hosting: Next.js static export + plain S3 static website hosting (not CloudFront)

Choice: `next.config.js` sets `output: 'export'`, producing a purely static build served directly from an S3 bucket's static website hosting endpoint (`aws_s3_bucket_website_configuration`), over plain HTTP, with no CDN in front of it.

Why: The original design planned S3 behind CloudFront with Origin Access Control, keeping the bucket itself fully private. Attempting that for real against this AWS Academy Learner Lab account surfaced a hard constraint: the account's `voc labs` IAM role denies `cloudfront:CreateOriginAccessControl`, the legacy `cloudfront:CreateCloudFrontOriginAccessIdentity`, and `cloudfront:CreateDistribution` itself — CloudFront distribution creation is unavailable in this account at any level, not just the origin-access sub-permissions. Rather than block the whole deployment on a service this account cannot provision, the frontend was pivoted to plain S3 static website hosting: a public-read bucket policy scoped to `s3:GetObject` only, with `block_public_acls/ignore_public_acls` left on (see `infra/frontend.tf` for the full inline explanation).

Trade-off: The live frontend is served over plain HTTP, not HTTPS, and has no CDN edge caching or global distribution — every request round-trips to us-east-1 regardless of visitor location. It also rules out any future Next.js feature that needs a server runtime (middleware, server actions) without a separate migration to Lambda@Edge or a Node runtime. In an unrestricted production account, this would revert to S3 + CloudFront + Origin Access Control, restoring HTTPS, edge caching, and a private origin bucket.

6. Implementation Summary

6.1 Repository Structure

`agents/` (one handler per negotiation agent), `api/` (the FastAPI/Mangum endpoint), `shared/` (web search, DynamoDB/S3 access, currency, logging), `data/` (static reference/fallback JSON), `infra/` (Terraform), `frontend/` (Next.js), `tests/` (pytest), and `docs/` (this report and its source diagrams). Full tree in the repository README.

6.2 Infrastructure as Code

Every AWS resource is defined in Terraform under `infra/`: ECR (the shared Lambda container image), five container-image Lambda functions plus one zip-based function, Step Functions, API Gateway (HTTP API), DynamoDB, two S3 buckets (one for reference data, one serving the frontend as a static website -- Section 5.8), Cognito, CloudWatch alarms + SNS, and an AWS Budgets guardrail. Because the app Lambdas reference a container image that doesn't exist until it's built and pushed, deployment is a documented three-step bootstrap (create the ECR repo → build/push the image → apply everything else) rather than a single `terraform apply` — see the README's Deployment section for the exact commands.

6.3 CI/CD Pipeline

`.github/workflows/deploy.yml` runs on every push/PR: install dependencies and run `flake8` + the full `pytest` suite; on a pull request, `terraform plan`; on a push to `main`, build and push the Lambda container image (matching the same bootstrap-ordering constraint as manual deployment) and then `terraform apply`. The `lint-test` job needs no AWS access and runs cleanly on every push; the `plan/apply` jobs authenticate with AWS Academy Learner Lab's temporary session credentials stored as GitHub Actions secrets, which expire on the order of hours, so a green `apply` run requires refreshing those secrets immediately before triggering the workflow. This project's actual AWS deployment (Section 9) was primarily driven by direct, local `terraform apply` runs for exactly this reason.

6.4 Testing

A `pytest` suite exercises the negotiation state machine and dynamic-data fetch/fallback logic; the frontend is checked with `tsc --noEmit` after every change. Individual features (age-based pacing, location autocomplete, exchange rates, the destination guide search) were each verified with targeted mocked tests exercising the actual handler functions end-to-end before being considered complete.

7. AWS Well-Architected Framework Mapping

Each subsection below states what's concretely implemented, cites the file it lives in, and — per the assignment's own allowance — explains anything that couldn't be implemented under AWS Academy Learner Lab's restrictions and how it would be done in an unrestricted production account.

7.1 Operational Excellence

- Structured logging throughout the agent handlers and API (`shared/logging_utils.py`).
- CloudWatch metric alarms on every Lambda's error count and p95 duration (alarmed at 80% of that function's configured timeout), plus a Step Functions execution-failure alarm, all notifying a shared SNS topic (`infra/monitoring.tf`).
- X-Ray active tracing enabled on all Lambda functions and the Step Functions state machine, giving an end-to-end trace of a single negotiation across every agent invocation in a round.
- API Gateway access logging to a dedicated CloudWatch log group.

7.2 Security

- Secrets: the Groq API key lives in SSM Parameter Store as a SecureString, never hardcoded (`infra/ssm.tf`).
- Network exposure: both S3 buckets block ACL-based public access; the reference-data bucket blocks all public access outright, while the frontend bucket is served via S3 static website hosting with a narrowly-scoped public-read bucket policy (`s3:GetObject` only) -- the CloudFront/Origin Access Control approach originally planned is unavailable in this AWS Academy Learner Lab account (Section 5.8).
- CORS: API Gateway and the FastAPI app's own CORS middleware are restricted to the real S3 static website endpoint (plain HTTP, no CloudFront domain) plus localhost, not a wildcard (`infra/apigateway.tf`, `api/main.py`).
- Authentication: a Cognito user pool, a public (no client-secret) SPA app client, and a JWT authorizer are fully provisioned and wired to `POST /trips` (`infra/cognito.tf`), gated behind `var.enable_auth` (default `true`). The frontend implements a real sign-up, email-confirmation, sign-in, and sign-out flow using SRP authentication via `amazon-cognito-identity-js` — no Hosted UI redirect required (`frontend/src/lib/auth.ts`, `frontend/src/components/LoginForm.tsx`). Once signed in, the SPA attaches the resulting Cognito ID token as a Bearer header on every API request via an axios interceptor (`frontend/src/lib/api.ts`); unauthenticated calls to `POST /trips` are rejected with 401 by the API Gateway authorizer itself, before reaching application code. This flow was exercised for real against the deployed stack: sign-up, email-code confirmation, sign-in, and an authorized `POST /trips` were all verified end-to-end (Section 9).

Learner Lab limitation: IAM relies on the Learner Lab's shared `LabRole` for every resource rather than per-function least-privilege roles, because Learner Lab accounts cannot create custom IAM roles/policies. In production, each Lambda would get a scoped role — e.g. the itinerary agent's role would only grant `dynamodb:PutItem/GetItem` on this one table and `ssm:GetParameter` on the Groq key, nothing else. Separately, local development against `local_api_server.py` intentionally skips the login screen (the local dev server never checks a JWT), so `frontend/.env.local` leaves the Cognito variables blank in that environment

only — deploys always populate them from Terraform outputs (`deploy-frontend.sh/.bat`).

7.3 Reliability

- The negotiation loop is hard-bounded at 3 rounds with a graceful force-finalize path — it can never hang, and unresolved objections are surfaced to the traveler rather than silently dropped.
- Every Step Functions Task state has Retry (3 attempts, exponential backoff) and Catch to a dedicated error-handling state (`infra/statemachine.asl.json`).
- DynamoDB point-in-time recovery is enabled, giving a 35-day restore window on the sole durable record of every negotiation.
- External dependencies degrade instead of crashing a round: web search falls back `ddgs` → `SearxNG` (`opt-in`) → `DuckDuckGo` scrape → static reference JSON, and travel-advisory search specifically prefers a dated news search over an undated one so stale results are labeled, not presented as current.
- Step Functions' `redrive-execution` capability was used for real during deployment testing to recover negotiations that failed at the Finalize step due to a since-fixed Supervisor Lambda packaging bug (Section 9) -- resuming from the failed state once the underlying code was corrected, rather than losing that execution's accumulated negotiation state.

7.4 Performance Efficiency

- DynamoDB runs in on-demand billing mode — no capacity planning, scales automatically with traffic.
- Lambda memory is tuned per function: 256MB for the lightweight Supervisor function, 512MB for the search/LLM-heavy agents and the API Lambda.
- Static frontend assets are served directly from the S3 static website endpoint with `Cache-Control: max-age=3600` on every object, so a repeat visitor's browser avoids re-fetching unchanged assets within the hour -- a partial substitute for the edge caching a CloudFront distribution would otherwise provide (Section 5.8).
- `arm64/Graviton2` compute across all Lambda functions (see Section 5.4).

7.5 Cost Optimization

- Pay-per-request billing for DynamoDB and API Gateway; no provisioned/idle capacity anywhere in the stack.
- `arm64/Graviton2` Lambdas (~20% cheaper than `x86_64` for equivalent work).
- S3 lifecycle rules expire noncurrent object versions after 30 days on both buckets, and abort incomplete multipart uploads after 7 days.
- ECR lifecycle policy expires untagged images after 7 days, preventing unbounded image storage growth from repeated `:latest` pushes.
- An AWS Budgets guardrail alerts at 80% (actual) and 100% (forecasted) of a \$5/month threshold.
- Groq's free tier keeps LLM inference cost entirely outside the AWS bill.

Learner Lab limitation: AWS Budgets is an account-level API that Learner Lab's LabRole commonly does not grant permission for. If `terraform apply` fails on the budget resource with an access-denied error, that is this exact, expected constraint — `infra/budgets.tf` documents it inline. In a production account, this budget would page the team well before the Learner Lab's own 45-credit ceiling could be exceeded.

7.6 Sustainability

- arm64/Graviton2 compute for every Lambda function is this project's primary concrete sustainability decision — AWS reports meaningfully lower energy use per request on Graviton2 versus x86_64 for equivalent Python workloads, which directly motivated Section 5.4's choice.
- The architecture is fully serverless (Lambda + Step Functions + on-demand DynamoDB) with zero idle compute between negotiations, unlike an always-on EC2/container-based design would have.
- S3 static website hosting has no idle compute of its own (unlike a CloudFront distribution's edge infrastructure, which this deployment does not use -- Section 5.8); the trade-off is that every request now travels the full distance to us-east-1 rather than being served from a nearby edge location, a real efficiency cost for visitors far from that region.

Learner Lab limitation: the deployment region is pinned to `us-east-1` (`infra/variables.tf`) because AWS Academy Learner Lab accounts are commonly restricted to a single region and cannot provision resources elsewhere. This is a real trade-off worth naming rather than ignoring: AWS's own sustainability guidance points to regions with a higher share of renewable/low-carbon grid power (for example `us-west-2` or `eu-west-1`) as having lower marginal carbon intensity per request than `us-east-1`. A production deployment without that regional restriction would weigh that against latency-to-users and data-residency requirements before picking a region.

8. AWS Services & Estimated Cost

Service	Usage	Cost Model
Lambda	api + 4 agents (container image, arm64) + supervisor	Pay-per-invocation
ECR	Shared Lambda container image	Storage + transfer; untagged images expire after 7 days
Step Functions	Negotiation orchestration	Standard workflow, pay-per-transition
DynamoDB	On-demand negotiation state, PITR on	On-demand billing
S3 (x2)	Reference data + frontend static website hosting	Standard + requests; lifecycle rules on old versions
API Gateway	HTTP API endpoint	Pay-per-request
Cognito	User pool (provisioned, enforcement on by default)	Free tier (50K MAUs)
CloudWatch + X-Ray	Alarms, 7-day log retention, traces	Mostly free-tier at this volume
SNS	Alarm/budget notifications	Free tier
SSM Parameter Store	Groq API key (SecureString)	Free (standard tier)
AWS Budgets	\$5/mo guardrail, 80%/100% alerts	Free (may be IAM-restricted in Learner Lab)
Groq (external, non-AWS)	Multi-round LLM calls	Pay-per-token; free tier available

No CloudFront line item appears above: the frontend is served directly from an S3 static website endpoint rather than through a CDN distribution (Section 5.8), which also means no CloudFront request/data-transfer cost in this deployment.

The Learner Lab credit budget is 45 credits. This project's live deployment (Section 9) has run well within that ceiling -- the stack is entirely pay-per-use serverless with no idle compute.

9. Deployment Status, Known Limitations & Remaining Work

Unlike an earlier draft of this report, which described a design that had not yet been applied to real AWS infrastructure, this project has since been deployed and exercised end-to-end against a live AWS Academy Learner Lab account. That process surfaced several real bugs that only manifest against actual AWS behavior -- the kind no amount of code review or local testing catches -- and each is recorded here rather than quietly fixed and forgotten, since the debugging process itself demonstrates real understanding of how these services actually behave in production.

9.1 Confirmed working end-to-end

- Every resource in `infra/` -- ECR, five container-image Lambdas plus the zip-based Supervisor, Step Functions, the HTTP API, DynamoDB, both S3 buckets, Cognito, CloudWatch alarms, SNS, and (IAM permitting) AWS Budgets -- is live in the account, created via `terraform apply` rather than merely planned.
- The frontend is deployed to its S3 static website endpoint and reachable over plain HTTP (Section 5.8).
- The full Cognito auth flow -- sign-up, email-code confirmation, sign-in, and an authorized `POST /trips` carrying a real Bearer ID token -- was exercised against the live deployment, not just unit tested.
- A complete multi-round negotiation (Itinerary → Budget → Logistics → Booking → CheckConvergence, looping on rejection) runs against real Step Functions, DynamoDB, and Lambda infrastructure, including agents making real, live-grounded Groq/ddgs calls.

9.2 Real bugs found and fixed via live deployment

- **API Gateway stage-prefix routing.** Mangum was not configured with `api_gateway_base_path`, so every request through the deployed `https://.../dev/...` URL arrived at FastAPI's router still carrying the `/dev` prefix, matching no route. This was masked during earlier testing because the one route exercised first (`POST /trips`) legitimately 401'd from the Cognito authorizer before ever reaching Lambda, which looked like a plausible, unrelated failure. Fixed in `api/main.py`.
- **Orphaned Lambda invoke permission on redeploy.** Forcing a container-image Lambda to pick up a newly-pushed `:latest` image requires `terraform apply -replace`, which destroys and recreates the underlying function object -- and AWS deletes that function's resource-based policy (the `lambda:InvokeFunction` grant for API Gateway) the moment it's destroyed. Because the permission resource's own arguments never change, Terraform had no reason to reapply it, silently leaving API Gateway unable to invoke the function (surfacing as a bare Internal Server Error with no Lambda logs at all). Fixed permanently with a `lifecycle.replace_triggered_by` rule on the permission resource (`infra/apigateway.tf`), not just a one-off reapply.
- **DynamoDB rejects native floats.** `boto3's Table.put_item` raises `TypeError: Float types are not supported` the first time a trip with a real numeric budget was submitted. Fixed with a recursive float-to-Decimal converter applied to every item written to DynamoDB (`shared/dynamo_utils.py`).
- **Supervisor Lambda missing a module in its zip bundle.** The Supervisor function's hand-picked zip file list (`infra/lambda.tf`) was never updated when `shared/logging_utils.py` was introduced,

so every real invocation failed at import time -- surfacing only once a negotiation actually reached the Finalize state, which simple/short negotiations reach immediately. Fixed by adding the missing source file to the archive.

- **Terraform silently skipping a zip-based Lambda's code update.** Even after fixing the bug above, `terraform apply` reported "No changes", because `aws_lambda_function.supervisor's filename` argument is just a local path string that never changes even when the zip's actual contents do. Fixed by wiring `source_code_hash` to the archive's own content hash (`infra/lambda.tf`), the same class of content-vs-identifier mismatch as the container-image :latest tag issue, but for zip archives.
- **Docker Buildx provenance/SBOM attestations.** Modern Buildx defaults to attaching build provenance/SBOM metadata on push, producing a multi-manifest image format that Lambda's container runtime rejects outright. Fixed with `--provenance=false --sbom=false` in both build scripts.
- **Stale Next.js build cache shipping the wrong auth state.** A production frontend build reused a `.next/` cache from earlier local development (where Cognito variables are intentionally blank to skip the login screen), so the deployed site silently shipped with authentication disabled. Fixed by forcing a clean `.next/` removal before every deploy build (`deploy-frontend.sh/.bat`).

9.3 Remaining / genuinely open items

- No formal load/performance testing has been conducted against the live deployment; the under-30s latency target in Section 3 is based on observed real-negotiation timing during manual testing, not a sustained load test.
- AWS Budgets may still be blocked by Learner Lab IAM depending on the specific account (see Section 7.5) -- this is an account-level restriction outside this project's control.
- CI/CD's `apply` job requires fresh AWS Academy Learner Lab session credentials in GitHub Secrets to succeed, since those credentials expire on the order of hours (Section 6.3); the actual deployment underlying this report was driven by direct local `terraform apply` runs for that reason.

10. AI-Assisted Coding Disclosure

The architecture of this system and every major technical decision were the author's: the multi-agent negotiation design, the AWS service choices and trade-offs discussed in Section 5, the security model in Section 7.2, and the resolution strategy for every real deployment blocker documented in Section 9 (what to fix, in what order, and how to verify the fix against live AWS infrastructure rather than assuming it worked) were all directed by the author.

An AI coding assistant (Claude) was used to carry out the implementation of that architecture under the author's direction: writing the negotiation agent handlers, shared utilities, Terraform infrastructure, and frontend code, and drafting this report's prose. The author reviewed each change, ran the system (both locally and against the real AWS deployment), read and diagnosed the actual CloudWatch logs, Step Functions execution history, and API Gateway responses behind every bug in Section 9, and directed the specific fix in each case. The author is responsible for the architecture, the major decisions, and the final verification that the system genuinely works end-to-end against live infrastructure; the AI assistant's contribution was code implementation carried out under that direction, not independent design authority.

11. Demo Plan (One-on-One Meeting)

- Submit a trip request deliberately over-budget (guarantees a Budget Agent rejection) and one with an unusual interest mix, to show genuinely tailored, per-interest itinerary content and a live round-2 revision.
- Show the live negotiation stream in the UI: round 1 rejection → round 2 revision → approval, each with its live sources.
- Open the DynamoDB table and the Step Functions execution graph side by side — same underlying data, two views.
- Trigger a CloudWatch alarm on purpose (e.g. temporarily invalidate the Groq key) to demonstrate the SNS notification path end-to-end.
- Open the X-Ray trace for one negotiation and point out which agent Lambda actually dominates round latency.
- Sign up a fresh account through the live login screen, confirm the email code, sign in, and submit a trip — showing the Cognito JWT actually gating `POST /trips`. Then name the remaining LabRole/IAM limitation explicitly and describe the production fix (Section 7.2).
- Close with the AWS Budgets guardrail and actual spend against the 45-credit Learner Lab ceiling.

12. References

- AWS Well-Architected Framework — <https://aws.amazon.com/architecture/well-architected/>
- Terraform AWS Provider — <https://registry.terraform.io/providers/hashicorp/aws/latest/docs>
- Groq API Documentation — <https://console.groq.com/docs>
- AWS Step Functions — <https://docs.aws.amazon.com/step-functions/>
- AWS Lambda container images — <https://docs.aws.amazon.com/lambda/latest/dg/images-create.html>
- AWS Graviton — <https://aws.amazon.com/ec2/graviton/>
- DynamoDB Design Patterns — <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/>